COMPUTER SCIENCE · EXAM-ORIENTED NOTES

Bitwise Operators

Bit Shifting • Left & Right Shift Operators • Their Uses & Limitations • Bitwise Relational Operators

Competitive Exams

Teacher Recruitment

C Language

§ Table of Contents

Introduction to Bitwise Operators 3

1. Concept of Modifying the Value Using Bit Shifting

- Bits and Bit Representation
- Binary Representation of Integers
- How Bit Shifting Modifies Values

2. Built-in Bit Shift Operators

- 2.1 Left Bit Shift Operator (<<)
- 2.2 Right Bit Shift Operator (>>)

3. Their Uses

4. Limitations of Bitwise Operators

5. Use of Bitwise Relational Operators

Quick Revision Section

Introduction to Bitwise Operators

Bitwise operators are special operators in C (and most other programming languages) that work directly on the individual **bits** of a value, rather than treating the value as a whole number. While ordinary arithmetic operators (`+`, `-`, `*`, `/`) manipulate numbers at the decimal level, bitwise operators manipulate the underlying binary representation stored in memory — turning individual bits on, off, or moving them to new positions.

Because a computer's memory ultimately stores everything as sequences of 0s and 1s, being able to operate directly on bits gives a programmer very fine-grained, efficient control over data. Bitwise operations are typically much faster than equivalent arithmetic operations, since they map to single, native CPU instructions.

1 Concept of Modifying the Value Using Bit Shifting

1.1 Concept of Bits and Bit Representation

Definition: A **bit** (binary digit) is the smallest unit of data in a computer, capable of holding only one of two values: **0** or **1**. Every value stored in computer memory — integers, characters, or floating-point numbers — is ultimately represented as a sequence of bits.

A group of bits forms larger storage units:

Unit	Number of Bits	Range of Unsigned Values
Nibble	4 bits	0 to 15
Byte (char)	8 bits	0 to 255
int (typical, 32-bit systems)	32 bits	0 to 4,294,967,295 (unsigned)

In C, the `int` type is most commonly **32 bits** on modern compilers. For simplicity and readability, most examples in this document use an **8-bit representation** of numbers, since the same shifting rules apply identically to 16-bit, 32-bit, or 64-bit integers.

1.2 Binary Representation of Integers

Every integer is stored internally as a binary (base-2) number. Each bit position represents a power of 2, counted from right (Least Significant Bit, LSB) to left (Most Significant Bit, MSB).

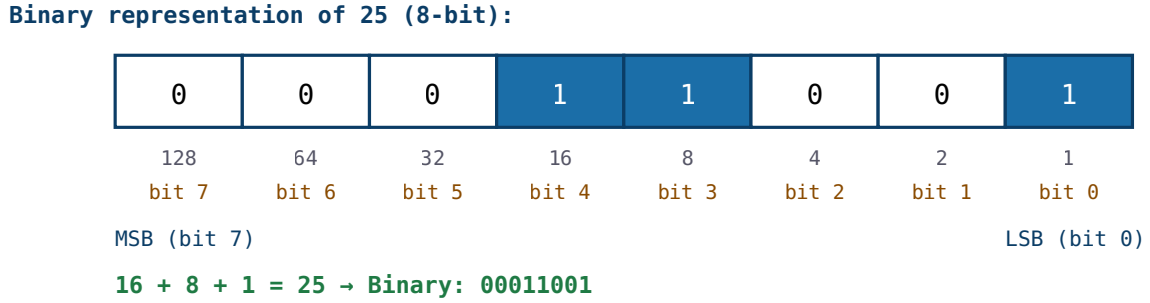


Figure 1.1 — 8-bit binary representation of the decimal value 25 (blue = bit value 1)

Note on Bit Numbering:

Bit positions are counted starting from **0**, not 1. So an 8-bit value has bit positions `bit 0` (LSB) through `bit 7` (MSB) — that is 8 positions in total (0,1,2,3,4,5,6,7), which correctly matches an 8-bit representation.

To convert decimal 25 to binary: 25 = 16 + 8 + 1 = $2^4 + 2^3 + 2^0$, so bits at positions 4, 3, and 0 are set to 1, giving `00011001`.

1.3 How Bit Shifting Modifies Values

Bit shifting is the process of moving every bit in a binary representation to the left or right by a specified number of positions. It is a fundamental low-level operation used to modify a value directly at the bit level, without arithmetic addition or subtraction.

> **Core Idea:**
> Shifting bits changes the "place value" each bit contributes to, which changes the overall numeric value — similar to how sliding digits left or right in a decimal number multiplies or divides it by powers of 10.

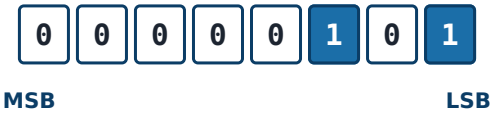
C provides two built-in operators for this purpose:

- **Left shift operator (`<<`)** — shifts bits toward the MSB (higher place value)
- **Right shift operator (`>>`)** — shifts bits toward the LSB (lower place value)

Step-by-Step Example

Consider the 8-bit value `00000101` (decimal 5). Shifting it left by 1 position:

1. Every bit moves one position to the left.
2. The MSB (leftmost bit) is discarded (shifted out).
3. A **0** is inserted at the vacated LSB (rightmost) position.



Before shifting: 00000101 = 5



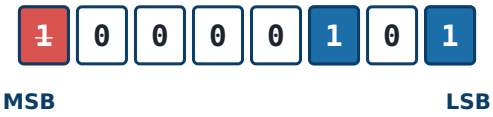
After `5 << 1` : 00001010 = 10 (grey cell = the newly inserted 0 at the LSB end)

This confirms that shifting left by one position doubled the value: 5 → 10. This relationship (**left shift by n = multiply by 2^n**) is explored in detail in Section 2.1 and Section 3.

What Happens When the Discarded Bit is 1 (Overflow Example)

In the example above, the bit discarded from the MSB end happened to be `0`, so no information was lost. But if the bit shifted out is `1`, that bit is permanently lost, and the result no longer equals the true mathematical value × 2 — this is called **overflow**.

Consider the 8-bit `unsigned char` value `10000101` (decimal 133). Shifting it left by 1 position:



Before shifting: 10000101 = 133 (red cell = the MSB, which is 1 and about to be discarded)



*After `133 << 1` (stored in 8 bits): 00001010 = 10 — **not** 266*

Why This Matters:

Mathematically, 133 × 2 = 266. But an 8-bit `unsigned char` can only hold values 0–255, so the bit representing 256 (the discarded leftmost 1) is thrown away, leaving 266 − 256 = 10. This silent loss of the shifted-out bit is exactly what Section 4 refers to as **overflow** — it happens with no error or warning, so the programmer must choose a data type wide enough to hold the expected result.

2 Built-in Bit Shift Operators

C provides two built-in bitwise shift operators. Both are **binary operators** (they require two operands): the value to be shifted, and the number of positions to shift by.

Operator	Name	Syntax	Effect
`<<`	Left Shift	`a << n`	Shifts bits of *a* left by *n* positions
`>>`	Right Shift	`a >> n`	Shifts bits of *a* right by *n* positions

2.1 Left Bit Shift Operator (<<)

Definition: The left shift operator `<<` shifts all bits of the left operand to the left by the number of positions specified by the right operand. Vacated bit positions on the right (LSB side) are always filled with **0**.

Syntax

```
result = value << number_of_positions;
```

Working Mechanism

- Every bit in *value* moves left by *n* positions.
- The *n* highest-order (leftmost) bits are discarded / shifted out.
- *n* zero bits are inserted at the rightmost (LSB) end.
- No bits "wrap around" from left to right — discarded bits are lost.

Bit-by-Bit Example

Let `a = 5` (00000101 in 8-bit form). Compute `a << 2`:

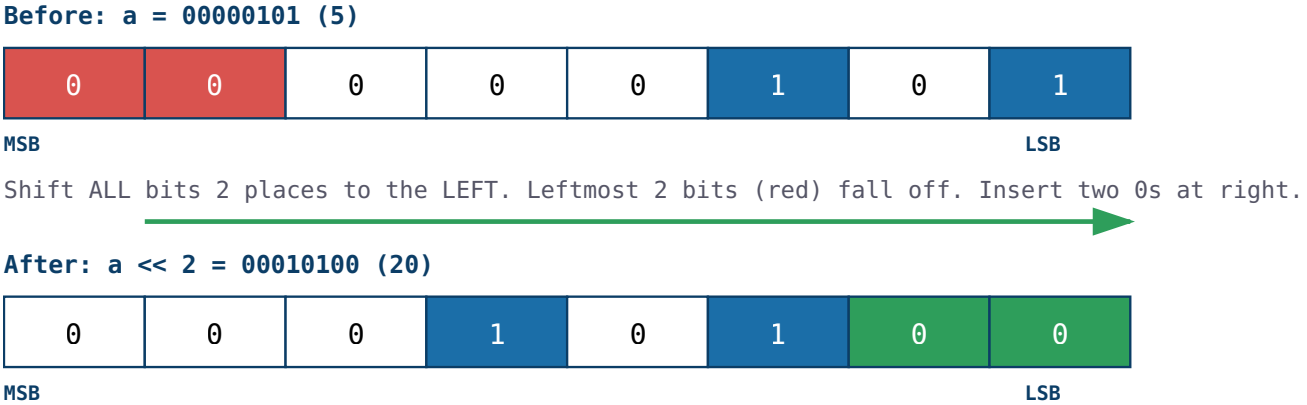


Figure 2.1 — Left shift by 2: bits move left, MSB side bits are discarded (red), zeros inserted at LSB (green)

```c
#include <stdio.h>
int main() {
    int a = 5;              // 00000101
    int result = a << 2;    // 00010100
    printf("%d", result);
    return 0;
}
```

Expected Output:

20

Effect on Positive and Negative Integers

For **positive** integers, left shift reliably multiplies the value by 2 for each position shifted, as long as no significant bit is lost (no overflow occurs).

Mathematical Example:

`7 << 3` = 7 × 2^3 = 7 × 8 = **56**

For **negative** integers, left shift is risky. In C, **left-shifting a negative number is undefined behaviour** — the result cannot be reliably predicted, so it should be avoided.

Example:

`int x = -2; x << 1;` → undefined behaviour (do not rely on any particular result)

Important Exam Point:

As per the ANSI C standard, if the left operand of `<<` is negative, or if the shift causes the result to overflow the representable range, the behaviour is **undefined**. Many textbooks/exams still ask you to "predict output" assuming two's complement wrap behaviour — always mention the undefined-behaviour caveat for full marks.

Applications of Left Shift

- Fast multiplication by powers of 2 (`x << n` instead of `x * pow(2,n)`)
- Setting a specific bit: `flags = flags | (1 << k)`
- Constructing bit masks for flags and permission systems
- Packing multiple small values into a single larger integer (e.g., RGB color packing)

Limitations and Important Rules of <<

- Shifting by a negative number of positions is **undefined behaviour**.
- Shifting by a number of positions **greater than or equal to the bit-width** of the type (e.g., `x << 32` for a 32-bit int) is **undefined behaviour**.
- Bits shifted out of the MSB end are permanently lost — left shift is **not** a rotate operation.
- Overflow can silently occur with no error or warning.

2.2 Right Bit Shift Operator (>>)

Definition: The right shift operator `>>` shifts all bits of the left operand to the right by the number of positions specified by the right operand. Bits shifted out at the LSB end are discarded; the fill value inserted at the MSB end depends on the data type (signed vs unsigned).

Syntax

```
result = value >> number_of_positions;
```

Working Mechanism

- Every bit in *value* moves right by *n* positions.
- The *n* lowest-order (rightmost) bits are discarded / shifted out.
- New bits are inserted at the MSB (leftmost) end — the value of these new bits depends on whether the shift is **logical** or **arithmetic**.

Logical vs Arithmetic Shifting (Relevance to C)

Type	Applies To	Fill Bit at MSB	Behaviour
Logical Right Shift	Unsigned types	Always **0**	Simple, predictable — equivalent to dividing by 2^n for magnitude
Arithmetic Right Shift	Signed types	Copy of the original **sign bit**	Preserves the sign of negative numbers (implementation-defined in C)

C Standard Note:

For an **unsigned** operand, `>>` is always a logical shift (0-fill), and this is well-defined by the C standard. For a **signed negative** operand, the C standard states that right shift is **implementation-defined** — most compilers (GCC, Clang, MSVC) implement arithmetic shift (sign-bit fill) on common hardware, but this is not guaranteed by the language specification.

Bit-by-Bit Example (Unsigned / Logical)

Let `unsigned char a = 20` (00010100). Compute `a >> 2`:

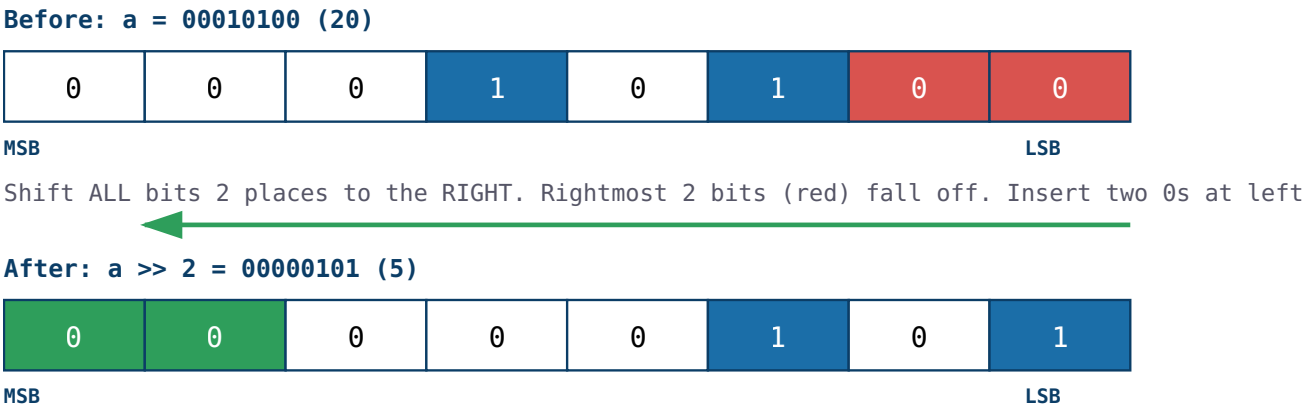


Figure 2.2 — Right shift by 2 on an unsigned value: zeros fill from the left (green), LSB bits discarded (red)

```c
#include <stdio.h>
int main() {
    unsigned char a = 20;      // 00010100
    unsigned char result = a >> 2;  // 00000101
    printf("%d", result);
    return 0;
}
```

Expected Output:

5

Bit-by-Bit Example (Signed / Arithmetic)

Let `int a = -8` (8-bit two's complement: 11111000). Compute `a >> 1`:

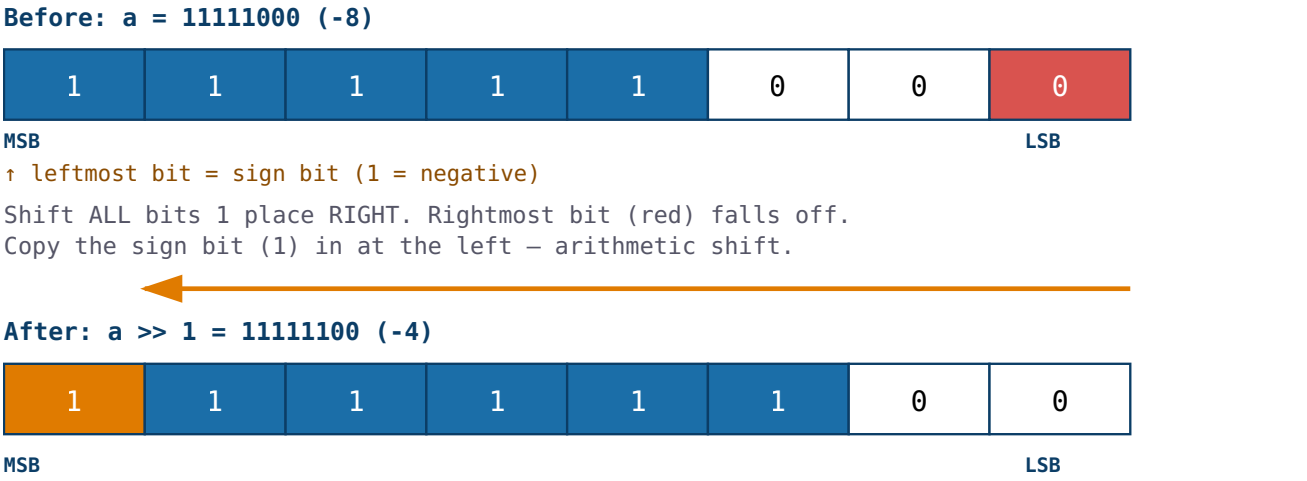


Figure 2.3 — Arithmetic right shift on a signed negative value: sign bit copied in (orange), LSB discarded (red)

Example with a Signed Negative Number (Arithmetic Shift)

```c
#include <stdio.h>
int main() {
    int a = -8;          // ...11111000 (two's complement)
    int result = a >> 1; // sign bit (1) copied in on most compilers
    printf("%d", result);
    return 0;
}
```

Typical Output (GCC/most compilers):

-4 (implementation-defined, not guaranteed by the C standard)

Applications of Right Shift

- Fast division by powers of 2 for non-negative values (`x >> n` instead of `x / pow(2,n)`)
- Extracting specific bits/fields from a packed value
- Checking individual bit values: `(x >> k) & 1`
- Reducing/scaling values in image processing and embedded systems

Limitations and Important Rules of >>

- Behaviour on **signed negative** numbers is **implementation-defined**, not guaranteed portable.
- Shifting by a negative count, or by a count ≥ the bit-width of the type, is **undefined behaviour**.
- Right shift of a negative number is **not** true integer division for all cases — rounding differs (right shift rounds toward negative infinity; integer division truncates toward zero) for negative operands.
- Bits shifted out at the LSB end are permanently lost — this is not a rotate operation.

Visual Comparison of << and >>

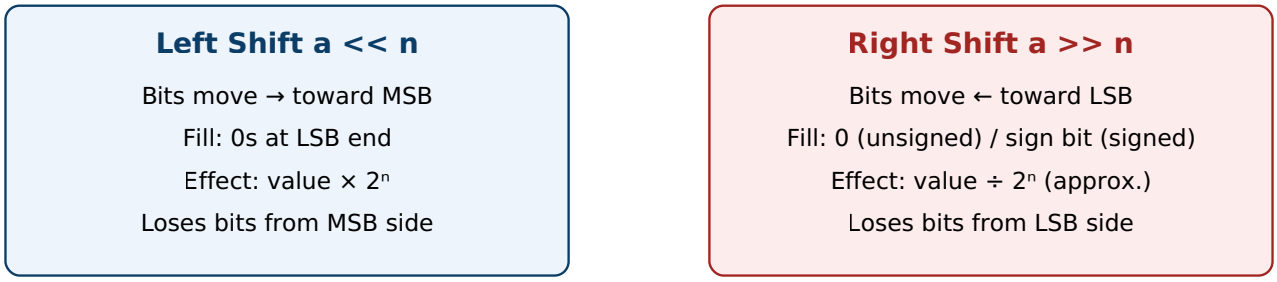


Figure 2.4 — Side-by-side comparison of left shift and right shift behaviour

3 Their Uses

3.1 Practical Uses of << and >>

Use Case	Operator	Example
Fast multiplication by 2^n	`<<`	`x << 3` → x × 8
Fast division by 2^n (non-negative)	`>>`	`x >> 3` → x ÷ 8
Setting a bit	`<<` with `|`	`x = x | (1 << 3)`
Testing/reading a bit	`>>` with `&`	`bit = (x >> 3) & 1`
Packing values into one integer	both	RGB colour packing
Extracting packed fields	`>>`	Unpacking a byte from a word

3.2 Multiplication / Division by Powers of 2

```
// Left shift = multiplication
int x = 7;
printf("%d", x << 1); // 14
printf("%d", x << 2); // 28
printf("%d", x << 3); // 56
```

Output:

```
14 28 56
```

```
// Right shift = division
int y = 40;
printf("%d", y >> 1); // 20
printf("%d", y >> 2); // 10
printf("%d", y >> 3); // 5
```

Output:

```
20 10 5
```

Exam Point:

`x << n` is mathematically equivalent to `x * (int)pow(2, n)` for non-overflowing positive x, and `x >> n` is equivalent to `x / (int)pow(2, n)` for non-negative x. These shift-based forms are computed faster because bit shifting is a single low-level CPU instruction, unlike general multiplication/division.

3.3 Bit Manipulation Examples

Example 1 — Setting a Bit

```
int x = 0;          // 00000000
x = x | (1 << 3);   // sets bit 3 → 00001000
printf("%d", x);
```

Output:

```
8
```

Example 2 — Checking a Bit

```
int x = 10;              // 00001010
int bit1 = (x >> 1) & 1; // checks bit at position 1
printf("%d", bit1);
```

Output:

1

Example 3 — Combined Shift Expression (Exam-Style)

```
#include <stdio.h>
int main() {
    int a = 6;                 // 00000110
    int b = (a << 2) >> 1;     // (6<<2)=24, 24>>1=12
    printf("%d %d", a, b);
    return 0;
}
```

Output:

6 12

3.4 More Exam-Oriented Examples and Outputs

Expression	Binary Working	Result
`3 << 4`	00000011 → 00110000	48
`100 >> 2`	01100100 → 00011001	25
`1 << 5`	00000001 → 00100000	32
`255 >> 4` (unsigned char)	11111111 → 00001111	15
`(unsigned char)(200 << 1)`	11001000 → 10010000 (overflow, bit lost)	144

4 Limitations of Bitwise Operators

4.1 Data Type Considerations

Bitwise shift operators work on the underlying bit pattern of a value, so their result depends heavily on the **size (width)** of the data type used:

- Shifting a `char` (8-bit) behaves differently from shifting an `int` (typically 32-bit) for the same numeric value, because the number of available bit positions differs.
- In C, operands smaller than `int` (like `char` or `short`) undergo **integer promotion** to `int` before the shift is applied, which can produce results different from what a programmer expects if they assume 8-bit behaviour.

4.2 Signed and Unsigned Integer Behaviour

Aspect	Unsigned Type	Signed Type
Right shift fill bit	Always 0 (logical) — well defined	Sign bit copied (arithmetic, typical) — implementation-defined
Left shift on negative value	Not applicable (no negatives)	**Undefined behaviour** in C
Overflow on left shift	Well-defined wraparound (modulo 2^n)	**Undefined behaviour** if sign bit is affected

Common Exam Trap:
Students often assume `-8 >> 1` always equals `-4` on every system. This is only true because most modern compilers implement arithmetic right shift — the C standard does **not** guarantee this. Correct exam answers should note "implementation-defined" for signed right shift.

4.3 Shift-Count Restrictions

- The shift count (right operand) must be **non-negative**. A negative shift count is undefined behaviour.
- The shift count must be **strictly less than the bit-width** of the promoted type. For a 32-bit `int`, shifting by 32 or more is undefined behaviour (it does NOT wrap around to a valid shift as some students assume).

Rule of Thumb:
For an *n*-bit type, a valid shift count *k* must satisfy **0 ≤ k < n**.

4.4 Portability and Implementation Considerations

- The size of `int` can differ across platforms (16, 32, or 64 bits), which changes shift results and valid shift ranges.

- Right shift of negative numbers is implementation-defined, so results may vary across compilers.
- Use `sizeof` or fixed-width types (`int32_t`, etc.) when exact bit-width matters.

4.5 Common Programming Mistakes

Mistake	Why It's a Problem
Shifting by a count ≥ type width	Undefined behaviour — may not produce 0 as expected
Left-shifting a negative number	Undefined behaviour as per the C standard
Assuming `x >> n` always equals `x / 2^n`	Fails for negative x due to rounding direction differences
Forgetting integer promotion of `char` / `short`	Result may differ from expected small-width behaviour
Confusing shift operators with rotate	Shift discards bits permanently; it does not wrap them around

5 Use of Bitwise Relational Operators

5.1 Explaining the Syllabus Concept

The syllabus term **"bitwise relational operators"** refers to the practice of using C's relational (comparison) operators — such as `==`, `!=`, `<`, `>` — **together with** bitwise shift/logic operators to test, compare, or validate the results of bit-level operations. It is not a separate category of operator; rather, it is the combined, purposeful **use** of relational operators on bitwise expressions.

> **Definition:** A bitwise-relational usage evaluates a bitwise expression (e.g. a shift or mask operation) and then applies a relational operator to test its outcome — commonly used to check whether a specific bit is set, cleared, or whether two bit patterns match.

5.2 Distinguishing Bitwise Operations from Relational Operations

Aspect	Bitwise Operators	Relational Operators
Symbols	`<< >> & | ^ ~`	`== != < > <= >=`
Operates on	Individual bits of operands	Whole numeric values (post bitwise evaluation)
Result type	An integer value (modified bit pattern)	A boolean-like result: 1 (true) or 0 (false)
Typical use	Modify or extract bits	Test/compare the outcome of an expression

5.3 Relevant Operators and Correct Usage

When combined with shift operators, the relational operators are used as follows:

- **Equality check** — verify if a shifted/masked value equals an expected pattern: `if ((x >> 2 & 1) == 1)`
- **Inequality check** — verify a bit is NOT in a certain state: `if ((x & mask) != 0)`
- **Magnitude comparison** — compare the numeric result of a shift against a threshold: `if ((x << 1) > 100)`

> **Operator Precedence Warning (Very Common Exam Trap):**
> In C, shift operators (`<<` , `>>`) have **lower precedence** than relational operators (`<` , `>`) but **higher precedence** than equality operators (`==` , `!=`). Always use parentheses to avoid ambiguity:
> `if (x >> 1 == 1)` is parsed as `if (x >> (1==1))` = `if (x >> 1)` — **NOT** what most programmers intend!
> Correct form: `if ((x >> 1) == 1)`

5.4 Examples with Binary Representation and Output

Example 1 — Checking if a Bit is Set

```
#include <stdio.h>
int main() {
    int x = 10;                    // 00001010
    if ((x >> 1 & 1) == 1)
        printf("Bit 1 is set");
    else
        printf("Bit 1 is not set");
    return 0;
}
```

Output:

Bit 1 is set

Example 2 — Comparing Shifted Value Against a Threshold

```
#include <stdio.h>
int main() {
    int x = 5;              // 00000101
    if ((x << 2) > 15)      // 5<<2 = 20, 20 > 15 → true
        printf("Greater");
    else
        printf("Not Greater");
    return 0;
}
```

Output:

Greater

Example 3 — Precedence Trap Demonstration

```
#include <stdio.h>
int main() {
    int x = 4;
    printf("%d", x >> 1 == 1);  // x >> (1==1) = x >> 1 = 2
    return 0;
}
```

Output:

2 (not the "1==1 first, then shift" result many students expect)

5.5 Common Exam Traps and Misconceptions

- **Misconception:** "`>>` and `>` can be mixed freely without parentheses." → Always causes precedence errors; parentheses are mandatory for correctness.
- **Misconception:** "Bitwise relational operators are a distinct operator category in C." → They are simply relational operators applied to the results of bitwise expressions; C has no separate "bitwise-relational" operator symbol.
- **Misconception:** "`==` and `&` behave the same for boolean-style checks." → `&` is a bitwise AND, not a truth comparison; `x & 1 == 1` is often mis-evaluated due to precedence.

✓ Quick Revision Section

Operator Syntax Table

Operator	Symbol	Syntax	Meaning
Left Shift	`<<`	`a << n`	Shift bits of a left by n, fill 0 at LSB
Right Shift	`>>`	`a >> n`	Shift bits of a right by n, fill 0/sign-bit at MSB

Bit-Shift Rules

Rule	Statement
Valid shift range	$0 \leq n <$ bit-width of type
Left shift effect	value $\times 2^n$ (if no overflow)
Right shift effect (unsigned)	value $\div 2^n$, floor, logical (0-fill)
Right shift effect (signed negative)	Implementation-defined; typically arithmetic (sign-fill)
Negative left operand + <<	Undefined behaviour
Shifted-out bits	Permanently lost (not rotated)

Important Binary Examples

Expression	Result
`5 << 1`	10
`5 << 2`	20
`20 >> 2`	5
`1 << 5`	32
`100 >> 2`	25
`-8 >> 1` (typical)	-4

Key Exam Facts

- Left shift by n = multiply by 2^n; right shift by n ≈ divide by 2^n.
- Right shift fill bit: 0 for unsigned, sign-bit copy (typical) for signed.
- Shift count must be $0 \leq n <$ bit-width; violating this is undefined behaviour.
- Left-shifting a negative number is undefined behaviour in C.
- Shift operators discard bits permanently — they are not rotate operators.
- Relational operators (`==, !=, <, >`) are used to test the results of shift/mask expressions — always parenthesize.

Common Mistakes (At a Glance)

- Forgetting parentheses when mixing shift and relational/equality operators.
- Assuming right shift always equals exact division for negative numbers.
- Shifting by ≥ bit-width and expecting a "wrapped" or zero result.
- Ignoring integer promotion of `char` / `short` operands before shifting.

? Frequently Asked Exam Questions

Q1. What is the effect of the left shift operator on an integer value?
It shifts all bits to the left by the specified number of positions, filling vacated LSB positions with 0, which multiplies the value by 2^n (for positive values, absent overflow).

Q2. Why is left-shifting a negative number undefined behaviour in C?
Because shifting can alter or corrupt the sign bit's meaning in two's complement representation, and the C standard does not define a consistent result for this case across implementations.

Q3. What is the difference between logical and arithmetic right shift?
Logical right shift always fills the vacated MSB positions with 0 (used for unsigned types). Arithmetic right shift fills them with a copy of the sign bit, preserving the sign of negative numbers (typical behaviour for signed types, though implementation-defined).

Q4. Is `x >> 1` the same as `x / 2` for all integers?
Not always. For non-negative x, they match. For negative x, right shift rounds toward negative infinity while integer division truncates toward zero, so results can differ.

Q5. What happens if the shift count equals or exceeds the width of the data type?
The behaviour is undefined by the C standard — the result is unpredictable and must never be relied upon in portable code.

Q6. Why must parentheses be used when combining shift and relational operators?
Because `>>` / `<<` have lower precedence than `<` / `>` but higher precedence than `==` / `!=`, expressions without parentheses are frequently parsed differently than intended.

Q7. Give one real use each for left shift and right shift.
Left shift: setting a specific bit using `flags |= (1 << k)`. Right shift: extracting/testing a specific bit using `(x >> k) & 1`.